APUNTES. ACLARACION DE CONCEPTOS O-O

Apuntes didácticos: public, private, protected, void, static

Objetivo: entender qué significa cada palabra clave y cuándo usarla.

Estos conceptos aparecen todo el tiempo en clases, métodos y propiedades en C#.

1) Modificadores de acceso: quién puede usarlo

Los modificadores de acceso indican desde dónde se puede acceder a un miembro (campo, propiedad, método) o incluso a una clase. Piensa en una "casa":

- public: cualquiera puede entrar
- private: solo la familia (la propia clase)
- protected: la familia y los hijos (clases hijas)
- internal: los vecinos del mismo edificio (mismo proyecto)

1.1) public

public significa: accesible desde cualquier parte del programa (si la referencia es visible).

```
public class Persona
{
    public string Nombre { get; set; }

    // Cualquiera puede leer/escribir Nombre
}
```

1.2) private

private significa: accesible SOLO desde dentro de la misma clase. Sirve para encapsular (proteger el estado).

```
public class CuentaBancaria
{
    private decimal _saldo; // Nadie fuera puede tocarlo directamente

    public void Ingresar(decimal cantidad)
    {
        if (cantidad <= 0) throw new ArgumentException("Cantidad inválida");
        _saldo += cantidad;
    }

    public decimal ObtenerSaldo() => _saldo;
}
```

1.3) protected

protected significa: accesible dentro de la clase y dentro de las clases que heredan de ella.
Se usa cuando quieres que las clases hijas puedan reutilizar algo interno, pero sin hacerlo público.

```
public class Persona
{
    protected string Dni; // Visible para Persona y para clases hijas
}

public class Alumno : Persona
{
    public void MostrarDni()
    {
        Console.WriteLine(Dni); // OK (protected)
    }
}
```

1.4) internal

internal significa: accesible dentro del MISMO PROYECTO (ensamblado). Es útil para "ocultar" código a otras librerías/proyectos.

```
internal class Motor
{
     // Solo accesible desde el mismo proyecto
}
```

1.5) protected internal y private protected

- protected internal: accesible desde el mismo proyecto O desde clases hijas (aunque estén en otro proyecto).
- private protected: accesible solo desde clases hijas, pero además deben estar en el mismo proyecto.

Son POCO USADOS.

2) void: métodos que no devuelven valor

Un método puede devolver un valor (int, string, bool…) o no devolver nada.

Si no devuelve nada, el tipo de retorno es void.

```
public void Saludar()
{
    Console.WriteLine("Hola");
}
```

En cambio, si devuelve un valor, debe usar return:

```
public int Sumar(int a, int b)
{
    return a + b;
}
```

3) static: pertenece a la clase, no al objeto

static significa: este miembro NO pertenece a una instancia (objeto) sino a la clase en sí.

• Miembro normal: necesitas new para usarlo.
• Miembro static: lo llamas con el nombre de la clase.

3.1) static en una propiedad/campo (dato compartido)

```
public class Jugador
{
    public static int TotalJugadores = 0;

    // Compartido por TODOS los jugadores
    public string Nombre { get; }

    public Jugador(string nombre)
    {
        Nombre = nombre;
        TotalJugadores++;
    }
}

// Uso:
var j1 = new Jugador("Ana");
var j2 = new Jugador("Luis");
Console.WriteLine(Jugador.TotalJugadores); // 2
```

3.2) static en un método (utilidades)

```
public static class Utilidades
{
    public static int TirarDado()
        => Random.Shared.Next(1, 7);
}

// Uso:
int n = Utilidades.TirarDado();
```

3.3) Cuidado con static

- Lo static se comparte: si alguien lo cambia, afecta a todos.
- Útil para constantes, utilidades, contadores globales, configuraciones compartidas.
- No conviene para datos que deben ser distintos en cada objeto.

4) const y readonly: datos que no deben cambiar

4.1) const

const crea una constante: debe asignarse al declarar y nunca cambia.

```
public class Matematica
{
    public const double Pi = 3.1415926535;
}
```

4.2) readonly

readonly permite asignar en la declaración o en el constructor. Después ya no se puede cambiar. Muy útil para dependencias internas.

```
public class Dado
{
    private readonly Random _rnd = new Random(); // se asigna una vez
    public int Tirar() => _rnd.Next(1, 7);
}
```

5) Propiedades: get; set; y variantes

En C# es común usar propiedades para exponer datos. Una propiedad puede tener get y set.

- get: leer el valor
- set: cambiar el valor

```
public class Persona
{
    public string Nombre { get; set; } // se puede leer y cambiar desde fuera
}
```

5.1) set privado (encapsulación)

Si no quieres que desde fuera cambien el valor, puedes hacer el set privado.

```
public class Jugador
{
    public string Nombre { get; private set; }
    public int Puntuacion { get; private set; }

    public Jugador(string nombre)
    {
        Nombre = nombre;
        Puntuacion = 0;
    }

    public void SumarPuntos(int puntos)
    {
        if (puntos < 0) throw new ArgumentException("Puntos inválidos");
        Puntuacion += puntos;
    }
}
```

6) virtual y override (relacionado con polimorfismo)

Cuando veas virtual y override, normalmente estás ante polimorfismo:

- virtual (en la clase base): permite que una clase hija cambie el comportamiento.
- override (en la clase hija): redefine ese comportamiento.

```
public class Vehiculo
{
    public string Marca { get; set; }
    public virtual void MostrarFicha()
        => Console.WriteLine($"Vehículo: {Marca}");
}

public class Coche : Vehiculo
{
```

```
    public override void MostrarFicha()
        => Console.WriteLine($"Coche: {Marca}");
}
```

Si en una lista tratas todo como Vehiculo, cada uno ejecuta su versión:

```
List<Vehiculo> lista = new()
{
    new Vehiculo { Marca = "Genérico" },
    new Coche { Marca = "Seat" }
};

foreach (var v in lista)
    v.MostrarFicha();
```

7) Resumen

- public: accesible desde cualquier parte.
- private: solo dentro de la clase.
- protected: clase y clases hijas.
- internal: solo dentro del mismo proyecto.
- void: método que no devuelve valor.
- static: pertenece a la clase (compartido), no al objeto.
- const: constante (no cambia nunca).
- readonly: se asigna una vez (declaración/constructor).
- get/set: propiedades para leer/cambiar datos (set puede ser private).
- virtual/override: polimorfismo (comportamiento distinto en clases hijas).

8) Ejercicio corto repaso

1) Crea una clase Contador con un campo private _valor y una propiedad pública Valor (solo get).

2) Crea un método public void Incrementar() y otro public void Reiniciar().

3) Crea 2 contadores diferentes y comprueba que cada uno mantiene su propio estado (no uses static).